

Verification and Validation Plan – RDM

Document ID	VVP-001
Revision	8569a89
Date	2026-10-03
Status	Controlled in git

Contents

1 Purpose	3
2 Intended use	3
3 User needs	3
4 Verification approach	4
5 Validation approach	4

Id: VVP-001

1 Purpose

Defines how RDM is verified (does it meet its design inputs, the acceptance criteria?) and validated (does it meet the user needs / intended use?). The vocabulary is the repository's

`CONTEXT.md`.

2 Intended use

RDM keeps the design record of regulated software — needs, design inputs, checklists, tagged tests, results and their git history — and lets the team gate releases on it, render regulatory documents from it, and query it as a read-only graph. Its users are regulatory authors, engineers, reviewers, and agents working under their direction. RDM is not a medical device and does not itself establish compliance; it keeps the evidence that a team presents.

3 User needs

Declared in this document's frontmatter (`user_needs`) — the validation anchors and the coverage denominator. Every user need must be validated and fully verified before a release.

- **UN-001** — A regulatory author can compile a Design History File from the system of record (SDD + executed tests + git).
- **UN-002** — A team is prevented from transitioning into implementation until the design input and design review are approved.
- **UN-003** — A release is blocked until every user need is verified by a passing test.
- **UN-004** — Each user need's verification status is traceable from executed test results.
- **UN-005** — The usability of a documented UI can be exercised formatively against a user need.
- **UN-006** — A regulatory author can check that documents contain the references a chosen standard/checklist requires (gap analysis).
- **UN-007** — A contributor is stopped when a user-need or design-input id is declared more than once, and told every document that declares it.
- **UN-008** — A regulatory author can scaffold a new compliant documentation project from a single command.
- **UN-010** — A contributor (human or agent) is guided to author a new design input that is fully traced: declared in its owning context, verified by a tagged test, and carried through the gates.
- **UN-011** — An existing repository (brownfield, little or no documentation) can be brought under record-first design controls from a single command, without disturbing its current contents.

- **UN-012** — A release’s evidence (verification data, traceability matrix) and the device-master-record index can be produced as retained, generated artifacts from the record.
- **UN-013** — A reviewer can prove, on demand, that a specific verifying test detects a specific defect in the code it verifies.
- **UN-014** — A regulatory author can query and visually explore the design record as a linked graph, without changing how the record is authored.
- **UN-015** — An agent, in any harness, can read the design record — query it, trace a need or input to its tests and results, and check it against the gate rules — and cannot change it.
- **UN-016** — A team can keep its risk register in the record, with each risk scored from its own matrix and each control traced to a verified design input, and cannot release while a risk is unscored, uncontrolled, or residually unacceptable without a named acceptance.
- **UN-017** — A team can keep its architecture in the record as C4 diagrams (system context and containers for the system, components for each bounded context) and is warned wherever the code, the tests and the diagrams disagree.
- **UN-018** — A user learns from RDM’s user manual, a controlled document of each release, how to keep the record the way that release’s gates enforce, and a reviewer can see which pages of the manual name what a change touches.

4 Verification approach

Each user need is refined into **design inputs** (declared in the per-context design documents, `kind: design`), owned by bounded contexts; a context serves the needs its inputs trace to. Verification is anchored on the design inputs (§820.30(f): output meets input). A design input is an acceptance criterion, a `shall` requirement on RDM or one of its bounded contexts: *baseline* when it follows from a user need alone, *risk-based* when it is allocated as a risk control. Each is verified by an automated test in RDM’s `tests/`, tagged `@allure.story("DI-...")` (“live BDD”: the test verifies the criterion, with no separate spec to drift). The test’s verification steps are its own, not criteria: a design input is accepted as a whole, and what must be accepted separately is a separate design input. A user need is met when it is validated and every design input that `traces_to` it is verified, wherever those inputs are owned; a risk-based criterion counts once it is verified and its risk’s residual is acceptable. `rdm story release-gate` enforces this.

5 Validation approach

User need	Summative (record of truth)	Formative (supporting)
UN-001..004	maintainer review that the compiled DHF, gates, and traceability meet the documented intent	dogfooding: RDM compiles its own DHF (this file set)
UN-005	review of persona-skill output against a real UI journey	<code>usability-persona</code> skill runs (<code>rdm story persona</code>)

User need	Summative (record of truth)	Formative (supporting)
UN-006	maintainer review that gap analysis flags real missing standard references against shipped checklists	dogfooding: <code>rdm gap</code> over RDM's own released docs
UN-007	maintainer review that a duplicated id stops the design gate and names every declaring document	dogfooding: the design gate on RDM's own DHF at every commit
UN-008	maintainer review that a scaffolded project builds a release and passes the relevant gap checklists	dogfooding: <code>fresh_release_test</code> builds an init'd project end-to-end
UN-010	maintainer review that a scaffolded design input lands fully traced (frontmatter entry, tagged stub test, checklist) and that the agent workflow runbook matches the enforced gates	dogfooding: <code>rdm story new-input</code> used against RDM's own DHF; agent sessions following <code>dhf/AGENT_WORKFLOW.md</code>
UN-011	maintainer review that an adopted repository ends up with the working control surface (DHF skeleton, runbook, hook, bootstrap, CI) and that nothing pre-existing was overwritten	trial adoption into a scratch copy of a real repository; <code>rdm adopt</code> acceptance test exercises the skip-not-overwrite contract
UN-012	maintainer review that a produced evidence bundle and DMR index are complete and agree with the record they were generated from	dogfooding: <code>rdm story dmr / evidence-bundle</code> run against RDM's own DHF and the worked example
UN-013	maintainer review that a probe reports a broken behavior as caught only when the test genuinely fails, and never leaves the code mutated	dogfooding: reviewers probing RDM's own tests during pull-request review
UN-014	maintainer review that the projected graph agrees with the record it was built from (needs, inputs, contexts, documents, tests, results, commits) and is explorable in a graph browser	dogfooding: RDM's own DHF projected, queried, and browsed in AWS Graph Explorer
UN-015	maintainer review that an agent harness connected to the server answers traceability questions about RDM's own record correctly and has no way to change it	dogfooding: agent sessions on RDM itself using <code>rdm graph mcp</code>
UN-016	maintainer review that the gate's risk findings match a manual review of the	dogfooding: RDM's own tool-risk register (<code>dhf/documents/risk/</code>) held to the release gate

User need	Summative (record of truth)	Formative (supporting)
	same register with the risk-analysis method	
UN-017	maintainer review that RDM's own C4 diagrams, rendered, match the code they name, and that each conformance warning on them points at a real disagreement	dogfooding: RDM's own architecture and design documents carry C1–C3, held to <code>rdm graph validate</code>
UN-018	maintainer review that the manual's pages teach what the gates enforce, and that each page the graph flags, or names for a changed design input, is one to re-read	dogfooding: RDM's own manual (<code>dhf/documents/user_manual.md</code>) projected and held to <code>rdm graph validate</code>

Formative evidence never gates release.